SNOG'S

DIALOGUE SYSTEM LITE

Developer Documentation

Version 1.0 | Unity 2021.3+ | Requires XNode

Free asset — Unity Asset Store

1. Introduction

Best for: small-to-mid games with branching dialogue and no save system or localization pipeline needed.

- 5 built-in nodes: Start, End, Line, Choice, SetVariable
- Global and GraphLocal variable scopes — Int, Float, Bool, String
- AND-logic conditions on choice options
- Typewriter reveal with per-node speed control
- IDialogueUI interface — plug in any custom UI

Upgrade path: *Graphs are 100% compatible with Pro. No migration required — just import and go.*

This document covers installation, quick start, every node and field, the variable and condition systems, the UI layer, the full DialogueRunner API, and extension points. A full Lite vs Pro feature comparison is in Section 1.2.

1.1 Requirements

- Unity 2021.3 LTS or newer
- XNode by Thor Brigsted — 10$ on the Asset Store or free on GitHub (Siccity/xNode)
  - Asset Store
  - GitHub
- TextMeshPro — included with Unity

1.2 Lite vs Pro — Full Feature Comparison

Use this as a quick reference before and after evaluating whether to upgrade.

Feature	Lite	Pro	Notes
Nodes	5	9	Pro adds Event, Wait, Jump
Condition logic	AND only	AND + OR	Pro: OR between groups, AND within
Variable scopes	2	3	Pro adds Persistent (PlayerPrefs)
Math operations on vars	Set only	5 operations	Inc, Dec, Multiply, Toggle
Actor portraits	—	✓	Emotions, slots, alpha fade

Localization	Keys stored	Full system	ILocalizationProvider + LocalizationTable
Typewriter audio	—	✓	Per-char clips, pitch variance
Inline tags	—	✓	<pause>, <speed>, <event>, custom
Text animation	—	✓	Wave, shake, rainbow, fade-in
Save / Load	—	✓	PlayerPrefs + file + JSON API
Analytics	—	✓	IDialogueAnalyticsProvider interface
Dialogue history log	—	✓	Rolling log, OnEntryAdded event
Live debugger window	—	✓	Variables + history at runtime
Graph validator	—	✓	Scans all graphs for authoring errors
Upgrade compatibility	—	100%	Lite graphs work in Pro unchanged

2. Installation

2.1 Importing

1. Install XNode from the Asset Store or GitHub.

2. Import Snog's Dialogue System Lite from the Asset Store.
3. If prompted, allow Unity to upgrade TextMeshPro essentials.
4. Confirm no compiler errors in the Console before continuing.

2.2 Project Layout

```
Assets/
  Snog's Dialogue System Lite/
    Runtime/
      Graph/          DialogueGraph.cs
      Nodes/          StartNode, EndNode, LineNode, ChoiceNode, SetVariableNode
      Runner/         DialogueRunner, DialogueRuntime, DialogueContext
      UI/             IDialogueUI, SimpleDialogueUI, Typewriter
      Variables/      VariableStore, Condition, ConditionEvaluator, ...
    Editor/
      Inspectors/     Custom node inspectors
      Utils/          GlobalVariableKeyUtility
    Demo/             Sample dialogue graph asset
```

3. Quick Start

3.1 Creating Your First Graph

1. In the Project window, right-click → Create > Snog > DialogueSystem > Dialogue Graph.
2. Double-click the asset to open it in the XNode graph editor.
3. Right-click the canvas → Snog > DialogueSystem > Lite > Start. Repeat for Line and End.
4. Connect Start.next → Line.input, then Line.next → End.input.
5. Select the Line node and fill in Speaker Name and Inline Text in the Inspector.

3.2 Setting Up the Scene

1. Create a Canvas (GameObject > UI > Canvas).
2. Build a line panel: Panel containing a TMP speaker text, TMP line text, and a Continue button.
3. Build a choices panel: Panel with a vertical Layout Group and a Button prefab for options.
4. Add an empty GameObject, attach SimpleDialogueUI, wire all references in the Inspector.
5. Add a second empty GameObject, attach DialogueRunner, assign SimpleDialogueUI to UI Behaviour.

3.3 Playing a Graph from Code

```
using SnogDialogue.Runtime;
using UnityEngine;

public class NPC : MonoBehaviour
{
    [SerializeField] private DialogueRunner runner;
    [SerializeField] private DialogueGraph   graph;

    private void OnInteract()
    {
        if (!runner.IsPlaying)
            runner.Play(graph);
    }
}
```

Tip: Call runner.Stop() to interrupt a conversation at any time. OnDialogueFinished fires whether the graph ends naturally or is stopped.

4. Node Reference

Every node is colour-coded in the XNode graph editor for quick recognition.

	Node	Tint	Purpose
	Start	#4CAF50	Entry point. Every graph needs exactly one.
	End	#F44336	Ends dialogue and hides the UI.
	Line	#2196F3	Displays one line of dialogue; waits for player to continue.
	Choice	#FF9800	Presents player choices with optional conditions.
	SetVariable	#9C27B0	Writes a value to a variable store immediately.

Note: A graph can also terminate when a non-End node's output port is left unconnected. Using an End node is recommended for explicit flow control and clean UI shutdown.

4.1 Start Node

Has one output port (next). Connect it to the first node of your dialogue. A graph with no Start node logs a warning and returns immediately when played.

4.2 End Node

Calls UI.Hide(), sets the next node to null, and terminates the runner coroutine. Fires OnDialogueFinished. You can place multiple End nodes in one graph for different conversation outcomes.

4.3 Line Node

Displays one line of dialogue via the active IDialogueUI implementation. Execution pauses until the UI calls the continue callback (i.e. the player clicks Continue).

Field	Type	Description
Speaker Name	string	Displayed in the speaker name text. Leave empty to hide the speaker box.
Inline Text	string (TextArea)	The dialogue line shown to the player.
Localization Key	string	Stored for future use. Not resolved in Lite — upgrade to Pro for full localization.
Tags	string[]	Metadata passed to LineUIOptions.Tags for use in custom IDialogueUI code.
Speed Multiplier	float (min 0.05)	Scales the Typewriter's base speed. 1 = normal, 2 = twice as fast, 0.5 = half speed.

Event Key	string	Optional label sent to DialogueRunner.OnLineStart and OnLineEnd when this line plays. Leave empty if you do not need to distinguish between lines.
Start Delay	float (min 0)	Seconds to wait after execution reaches this node before firing OnLineStart. Default: 0.
Auto Advance	bool	If true, the line advances automatically after Auto Advance Delay seconds without waiting for player input.
Auto Advance Delay	float (min 0)	Seconds to wait before auto-advancing. Only used when Auto Advance is enabled. Default: 1.5.

Tip — Variable Substitution in Line Text: Use `{var:key}` tokens anywhere in Inline Text to insert a runtime variable value. Graph-local scope is checked first, then global. Example: `"Hello, {var:playerName}! You have {var:gold} gold."` If the key is not found at runtime, `[key?]` is shown and a warning is logged to the Console.

4.4 Choice Node

Presents a list of player choices and waits for a selection. Each option has its own output port. An optional Fallback port is used when all options are locked or the list is empty.

Field	Type	Description
Locked Choice Display	enum	Hide: locked options are invisible. Disable: shown at 50% alpha with a suffix.
Locked Suffix	string	Appended to disabled option labels. Default: " (Locked)".
Options	List<ChoiceOption>	Each option: Inline Text, Localization Key, Tags, Conditions array.
Fallback	output port	Followed when all options are hidden/locked or the list is empty.

Tip: Use the Add Option / Remove Last buttons in the Inspector. The ▲▼ arrows reorder options and swap their port connections automatically.

4.5 SetVariable Node

Writes a DialogueValue to a variable store and continues immediately — no UI interaction.

Field	Type	Description
Scope	enum	Global: session-wide. GraphLocal: cleared on next Play(). See Section 5 for when to use each.

Key	string	Variable name. A dropdown shows keys from any GlobalVariablesAsset found in the project.
Value	DialogueValue	The value to write. Select the Type first, then fill in the corresponding field.

5. Variable System

5.1 Scopes — When to Use Each

Use this table as your reference whenever you're deciding which scope to write to.

Scope	Lifetime	Reset trigger	Best for
Global	Entire play session	Manual or game restart	Cross-graph flags, quest state, counters
GraphLocal	One Play() call	Next Play() call	Temporary state local to one conversation

> **Note:** Persistent scope (survives game restarts via PlayerPrefs) is a Pro-only feature.

5.2 Value Types

Type	Best for
Int	Counters, quest stages, item quantities
Float	Timers, reputation scores, probability thresholds
Bool	Visited flags, quest accepted/completed, unlock gates
String	Player name, faction choice, free-text memory

5.3 GlobalVariablesAsset

Create one via Create > Snog > DialogueSystem > Variables > Global Variables Asset. Define default starting values here and assign the asset to DialogueRunner. A runtime copy is made at the first Play() call — the asset itself is never mutated at runtime.

5.4 Reading and Writing from Code

```
// Typed convenience methods — null-safe before first Play()
runner.SetGlobalBool("questAccepted", true);
runner.SetGlobalInt("gold", 100);

bool accepted = runner.GetGlobalBool("questAccepted");
int  gold     = runner.GetGlobalInt("gold", fallback: 0);

// Direct store access
runner.GlobalVariables.Set("playerName", DialogueValue.FromString("Snog"));
runner.GlobalVariables.TryGet("playerName", out DialogueValue nameVal);
```

```
// Debug dump to Console — useful during development
runner.GlobalVariables.DebugDump("Global");
```

6. Condition System

Conditions gate which Choice options are visible or interactable at runtime. They are configured per-option in the ChoiceNode Inspector.

6.1 Condition Fields

Field	Values	Description
Scope	Global / GraphLocal	Which variable store to read from.
Key	string	The variable name to evaluate.
Operator	see below	How to compare the stored value against Expected Value.
Expected Value	DialogueValue	The right-hand side of the comparison.

6.2 Comparison Operators

Operator	Behaviour
Exists	True if the key is present, regardless of value.
NotExists	True if the key is not present.
Equals	Stored value equals expected. Floats use Mathf.Approximately.
NotEquals	Stored value does not equal expected.
GreaterThan	Numeric only. Stored > expected.
GreaterOrEqual	Numeric only. Stored >= expected.
LessThan	Numeric only. Stored < expected.
LessOrEqual	Numeric only. Stored <= expected.

6.3 AND Logic

All conditions on a single option are evaluated with AND logic — every condition must pass for the option to appear. For OR logic between condition groups, upgrade to Pro.

> **Tip:** Example: option visible only if questAccepted == true AND gold >= 50. Add both as separate Conditions on the same option.

7. UI System

7.1 IDialogueUI Interface

The runner communicates with all UI through this interface. Implement it on any MonoBehaviour for full control over layout, animation, and input handling. SimpleDialogueUI is a reference implementation, not a requirement.

```
public interface IDialogueUI
{
    // Called when a Line node executes.
    // Call onContinueRequested when the player advances.
    void ShowLine(string text, LineUIOptions options, Action onContinueRequested);

    // Called when a Choice node executes.
    // Call onChoiceSelected(index) when a choice is made.
    void ShowChoices(IReadOnlyList<ChoiceUIEntry> choices, Action<int>
onChoiceSelected);

    // Called at dialogue end and when Stop() is called.
    void Hide();
}
```

LineUIOptions carries: SpeedMultiplier (float), SpeakerName (string), and Tags (string[]). Ignore any fields not needed by your implementation.

7.2 SimpleDialogueUI — Inspector References

Field	Description
Line Panel	Root GameObject shown during lines; hidden during choices.
Choices Panel	Root GameObject shown during choices; hidden during lines.
Line Text	TMP_Text for dialogue content.
Speaker Text	TMP_Text for the speaker name. Hidden automatically when name is empty.
Continue Button	Advances dialogue or skips the typewriter on first click, then advances on second.
Choices Container	Transform (with Layout Group) where choice buttons are instantiated.
Choice Button Prefab	Button prefab. Must have a TMP_Text child for the label.
Typewriter	Optional. Assign for animated text reveal.

7.3 Typewriter — Settings

Field	Description

Characters Per Second	Base reveal speed. Multiplied per-line by the Line node's Speed Multiplier.
Punctuation Multiplier	Pause length multiplier after . , ! ? : ; — default 6×.
Newline Multiplier	Pause length multiplier after line breaks — default 4×.
Use Unscaled Time	If true, typing ignores Time.timeScale. Useful for pause menus.

Tip: First click on Continue while typing calls SkipToEnd() — instant reveal. Second click advances to the next node.

8. DialogueRunner API

8.1 Inspector Fields

Field	Description
UI Behaviour	MonoBehaviour implementing IDialogueUI. Required.
Global Variables Asset	ScriptableObject with default variable values. Optional.
On Dialogue Started	UnityEvent fired at the beginning of Play().
On Dialogue Finished	UnityEvent fired when the graph ends naturally or Stop() is called.
On Line Start	UnityEvent<string> fired when a Line Node begins displaying. The string argument is the Event Key set on that node (empty string if none). Wire any scene script method here — no code required.
On Line End	UnityEvent<string> fired when the player continues past a Line Node. The string argument is the Event Key set on that node (empty string if none).

8.2 Methods

Method	Description
Play(graph)	Starts playing the graph. Interrupts any currently playing graph first.
Stop()	Stops playback, hides UI, fires OnDialogueFinished. No-op if nothing is playing.
SetGlobalBool(key, value)	Writes a bool. Null-safe before first Play().
SetGlobalInt(key, value)	Writes an int.
SetGlobalFloat(key, value)	Writes a float.
SetGlobalString(key, value)	Writes a string.
GetGlobalBool(key, fallback)	Returns the stored bool or fallback if the key is missing.
GetGlobalInt(key, fallback)	Returns the stored int or fallback.
GetGlobalFloat(key, fallback)	Returns the stored float or fallback.
GetGlobalString(key, fallback)	Returns the stored string or fallback.

8.3 Properties

Property	Description
IsPlaying	True while a dialogue coroutine is active.
GlobalVariables	The session-level VariableStore. Null before first Play().
GraphVariables	The graph-local VariableStore. Null before first Play().

8.4 Runtime Warnings

Warning	Cause
Play() called with a null graph	Null reference passed to Play().
No StartNode found in graph	Graph asset has no Start node.
Node has no outgoing connection	A non-End node's output port is disconnected.
Possible infinite loop detected	More than 2000 nodes executed — likely a cycle.

9. Extending the System

9.1 Custom Nodes

Extend DialogueNode and implement Execute(). Use XNode attributes for the node's appearance and right-click menu placement.

```
using System.Collections;
using SnogDialogue.Runtime;
using XNode;

[NodeTint("#009688")]
[NodeWidth(220)]
[CreateNodeMenu("MyGame/Dialogue/Heal Player")]
public sealed class HealPlayerNode : DialogueNode
{
    [Input(connectionType = ConnectionType.Override,
           backingValue = ShowBackingValue.Never)]
    public int input;

    [Output] public int next;

    public override IEnumerator Execute(DialogueRuntime runtime)
    {
        PlayerHealth.Instance.Heal(50);
        runtime.SetNext(GetNextFromPort("next"));
        yield break;
    }
}
```

Note — XNode GetValue() Warning: XNode calls `GetValue()` on every port when generating graph tooltips. If you see a “No GetValue() override defined” warning per node, add this one line to `DialogueNode` (or to your custom node class) to silence it for all nodes at once: `public override object GetValue(XNode.NodePort port) => null;`

9.2 Custom UI

Implement IDialogueUI on any MonoBehaviour and assign it to DialogueRunner's UI Behaviour field. You have complete control over layout, animation, and input. SimpleDialogueUI is the reference implementation.

9.3 Accessing Variables from Other Systems

```
// From a quest system
runner.SetGlobalBool("mainQuest_started", true);

// From a save system
bool talked = runner.GetGlobalBool("talkedToMerchant", fallback: false);

// Debug all current variables
runner.GlobalVariables.DebugDump();
```

10. Upgrading to Pro

Upgrading to Pro? Your graphs are 100% compatible. No graph rebuilding, no migration steps. Import Pro, delete the Lite folder, and your existing dialogue works immediately.

Pro is a full-featured paid upgrade with actors, localization, audio, save/load, analytics, three new nodes, OR conditions, and editor tooling. The full feature comparison is in Section 1.2.

10.1 Migration Steps

- Back up your project before proceeding.
- Delete the Lite folder (Assets/Snog's Dialogue System Lite/) from your project.
- Import Snog's Dialogue System Pro from the Asset Store.
- Your DialogueGraph assets open immediately — no rebuilding required.
- Pro's new inspector fields (Actor Database, Localization Table, etc.) default to empty/off. Assign them only when ready to use those features.

Appendix: File Reference

File	Folder	Responsibility
DialogueGraph	Runtime/Graph	XNode NodeGraph subclass. Holds a GUID and the node list.
DialogueNode	Runtime/Nodes	Abstract base. Provides NodeId and GetNextFromPort().
StartNode	Runtime/Nodes	Entry point. Green tint (#4CAF50).
EndNode	Runtime/Nodes	Terminates dialogue, calls UI.Hide(). Red tint (#F44336).
LineNode	Runtime/Nodes	Displays one dialogue line. Blue tint (#2196F3).
ChoiceNode	Runtime/Nodes	Player choices with conditional gating. Orange tint (#FF9800).
SetVariableNode	Runtime/Nodes	Writes to a variable store. Purple tint (#9C27B0).
DialogueRunner	Runtime/Runner	MonoBehaviour entry point. Play(), Stop(), variable access.
DialogueRuntime	Runtime/Runner	Non-MonoBehaviour state passed into every Execute() call.
DialogueContext	Runtime/Runner	Holds GlobalVariables and GraphVariables stores.
IDialogueUI	Runtime/UI	Interface contract for all UI implementations.
SimpleDialogueUI	Runtime/UI	Reference IDialogueUI — Unity Canvas with TMP.
Typewriter	Runtime/UI	TMP character-by-character reveal, skip support.
Condition	Runtime/Variables	Serializable: scope, key, operator, expectedValue.
ConditionEvaluator	Runtime/Variables	EvaluateAll(): AND logic across a Condition array.
DialogueValue	Runtime/Variables	Tagged union: Int, Float, Bool, String.
GlobalVariablesAsset	Runtime/Variables	ScriptableObject defining default variable values.
VariableStore	Runtime/Variables	Runtime dictionary: Has / TryGet / Set / Clear / DebugDump.